NanoMAP Data Analysis Workflow

1. **Use Imaris File Converter to import raw data as .ims**
    a. In File Converter, set the voxel size.
        i. Specify folder for saving converted files, or same folder as input
        ii. Select the file in File Converter, click "Set Voxel Size"

2. **Crop ROIs and save as .ims**
    a. Crop ROIs manually.
        1. Note:
            1. The ROI should contain the entire cell with some margines. Do not make the margin unnecessarily big, which will increase the files size and slow down Imaris processes.
            2. Try to keep using the **same ROI size for all your files, or at least for files in the same folder**. Different ROI sizes affect batch background subtraction results!

3. **Backup all channels with original intensity**
    a. Keep a copy of the .ims file as "pre-processed" data.
    b. This allows later comparison of raw image with background subtracted images.

4. **Background subtraction for every channel and cell detection**
    a. Run Imaris built-in Background Subtraction with default parameters
    b. Run Imaris built-in cell detection (on membrane channel) with optimized parameters

5. **(Optional) Sum normalized signals from all channels**
    a. Normalize and sum Ch1, Ch2 and Ch3. Can be used to boost membrane signals for cell detection.
    $$Channels\ Sum = \frac{32760}{3} * \sum_{i=1}^{3} \frac{ch_i - \max(ch_i)}{\max(ch_i) - \min(ch_i)}$$

6. **Check and optimize cell detection**
    a. Check cell detection. Modify if needed.
    b. For bad cell detection, redo cell detection:
        1. Use membrane channel --> if this does not work well, try 2
        2. Use All Channel Sum channel → if this does not work well, try 3
        3. Use one of the receptor channels with good signals for membrane segmentation
    c. Make sure the cell object does NOT contain multiple cells.
    d. After finalizing the cell segmentation, clean out other cell detection and keeping only **ONE** cell object.
    e. Export the cell to surface.
    f. Save files.

7. **Define the Cell Region and the MV Region → Code: *XTDistanceTransformInAndOut.m***
    a. Steps:
        i. Calculate distance transformation (DT) in and out of the cell surface
            1. Note: The XT checks for surface objects with **names contain** "*Cell Export*", and process the **first surface** if multiple surfaces are found with such name.
        ii. Create a new channel (**Distance to Cell**) from DT_Out – DT_In

iii. Create new surface objects:
1. "Cell Region": from Channel "Distance to Cell" with threshold, e.g. (-0.5, 1.5)
2. "MV Region": from Channel "Distance to Cell" with threshold, e.g. (0.15, 1.5)

iv. Mask all data channels with "Cell Region", "MV Region"
1. The code looks for channels that are background subtracted. Usually ch1, ch2, and ch3.
2. Generate new channels with names like "ch1_name Cell Region", "ch1_name MV Region".

b. **Run this XTension in Imaris**:
i. Distance Transformation In and Out: create distance transformation channel both in and out of the cell.

8. MV enrichment analysis → Code: *MV_enrichment_analysis.ipynb*
a. Export the data in the Imaris file as TIFF files to a new folder. Imaris exports TIFF files follow this naming convention: *"{sample_name}_C0{channel}_Z000{z_slice}.tif"*
b. Run Python script: **MV_enrichment_analysis.ipynb**
i. Set channel information
ii. Set range of z-slices for analysis
iii. Set other parameters following the instructions in the script.

9. Create surfaces for receptors from masked receptor channels (in the Cell Reigin)
a. Name the surface with the name of the receptor:
1. e.g. TCR, CD8 (**DO NOT** use H57, CD8a, rCD3)

10. Create MV from MV Region channel
a. Name the surface with MV: e.g. MV or WGA_MV
b. Further filter surface with DT_max, the resulting new surface is used as MV in downstream analysis.

11. Create masks for receptor clusters and MVs as needed
a. Set voxel intensity outside surface = 0, voxel intensity inside surface = 1

12. Receptor clusters in MV:
a. For clusters detected for each receptor (surface object), select for subsurfaces colocalize with MV (based on different coloc threshold)
i. Example 1: filter clusters by *Intensity Max in the DT channel* > a certain threshold (e.g. 0.2 um).
ii. Example 2: filter clusters by *Volume in MV Mask channel > threshold*
b. Duplicate Selection to new Surfaces
c. The receptor-clusters-in-MV surface has the name: *receptor Selection […….]*

13. Create channels for surfaces and label sub-surfaces with their index → Code: *XTSurfaceIndexMask.m* , *SurfaceIndexMask.m* , *align2Fullmask.m* , *Submask2Fullmask.m*
a. Run this **XTension** in Imaris**: Surface Index Mask**
i. Select one or multiple surface objects for analysis.
ii. For each selected surface object, creates a new channel "Index Mask ~"
iii. Saves vId_Mask.mat and vSurfaceList.mat in the data folder
b. Steps of the XTension:
i. Takes a list of selected surface objects
ii. Generate corresponding masked channels for each surface object.

iii. Each voxel in a subsurface takes the value of the ID of the subsurface. (Voxel value =0.1 for surface ID=0)
iv. Calls functions **SurfaceIndexMask.m, align2Fullmask.m, Submask2Fullmask.m**

14. Nearby Surfaces → Code: *XTNearbySurfaces.m* , *distantTransformationSubVolume.m*

a. Run this **XTension** in Imaris**: Nearby Surfaces**
b. Get information for nearby surfaces with a given range away from the edge of the subsurface.
c. Input:
    1. Primary Surface (select ONE)
    2. Secondary Surfaces (can select multiple)
    3. Set the distance range, e.g. 0.3 um.
d. Output:
    1. Saves *dataName_Surface1_Surface2_nearbySurface.csv* to the *coloc_result* folder.

15. Export statistics in Imaris

a. Export **all** statistics as .csv files to a result folder for the objects:
    1. Cell
    2. Cell membrane
    3. MV
    4. Receptors
    5. Receptor selections (e.g. receptors in MV)
b. Keep a record of the channels by taking a snapshot of the channel numbers in Imaris.

16. Characterization of receptor clusters, MV and colocalization analysis → Code: *MV_and_Clusters_parameter.ipynb* , *MV_and_Clusters.ipynb*

a. Run Python scripts:
    i. **MV_and_Clusters_parameter**
    ii. **MV_and_Clusters_analysis**
b. What these scripts do:
    i. Load statistics exported from Imaris of the given dataset. Data file required:
        1. *receptorA_All .xls*
        2. *receptorB_All.xls*
        3. *MV_select.xls*
        4. *receptorA_select.xls* (receptorA in MV)
        5. *receptorB_select.xls* (receptorB in MV)
    ii. Calculate properties and colocalization for receptors and MV, and plot the results.
    iii. Save the results as excel files:
        1. ***df_data_loaded.xlsx***
        2. ***check_sheet_name.xlsx***
        3. ***MV_cluster_analysis.xlsx***

17. Receptor distribution on a microvillus → Code: *NearbySurf_analysis.ipynb* , *NearbySurf_analysis_parameter.ipynb*

a. Run Python scripts:

   i. **NearbySurf_analysis_parameter**
   ii. **NearbySurf_analysis**

b. What these scripts do:
   i. Load files outputted from Imaris XT Nearby Surfaces and MV_and_Clusters analysis (Step 16):
      1. *dataName_MV_Surface1_NearbySurfaces.csv*
      2. *dataName_MV_Surface2_NearbySurfaces.csv*
      3. *df_data_loaded.xlsx*

      Note: Make sure that the files from the ImarisXT Nearby Surfaces were calculated based on selected receptor clusters with DT_max>0.2.
   ii. Categorize microvilli based on receptor occupancy and calculate number of receptor clusters per microvillus.
   iii. Select the cluster information from the MV analysis (Step 16) for all valid microvilli IDs and receptor clusters contained within microvilli.
   iv. Calculate distance ratios of receptor clusters:
      1. Each receptor cluster distance as compared to microvilli length.
      2. Taking the maximally located receptor cluster in each double positive microvillus and calculating the receptor1:receptor2 distance ratio.
   v. Calculate voxel overlap of both receptors in double-positive microvilli.
   vi. Save the results in an Excel file:
      1. ***Dataname_NearbySurf_analysis3.xlsx***

18. Cell score of receptor preference for tips of microvilli → Code: *Cell_Score_set-parameters.ipynb*, *Cell_Score_analysis.ipynb*

a. Run Python scripts:
   i. **Cell_Score_set-parameters**
   ii. **Cell_Score_analysis**

b. What these scripts do:
   i. Loads files from NearbySurf_analysis (Step 17):
      1. *dataName_NearbySurf_analysis3.xlsx*

c. Calculates a per-cell microvilli tip preference for Receptor 1 clusters over Receptor 2 clusters and reports a score value.
   i. Every microvillus in a cell is taken into consideration for the overall score calculation (see analysis notebook for full description of the score calculation).

d. Save the results as excel files:
   i. **MV_for_scoring.xlsx**
   ii. **folder_results.xlsx**